Jinx: Quiet, Explainable Duplicate Warnings for Pull Requests from Coding Agents

Harshul Jain
Independent Researcher

Saurabh Yergattikar
Independent Researcher

Tanya Sah
Independent Researcher

Abstract

Coding agents open many pull requests (PRs), and some of them repeat work that is already open in the same repository. In the AIDev dataset, 17% of duplicate PRs from different authors were noticed by a person only after more than a day, and in 54% of cases someone had already reviewed or commented on the PR that was later closed. Jinx is a GitHub Action and command-line tool that compares each new PR with the repository's open PRs and posts one comment listing likely duplicates, with a plain-language reason for each match (same files, identical lines, same issue, similar description). It runs on the CI runner's CPU, needs no API key and sends no code to an outside service. On 313 cross-author duplicate pairs scored against the PRs that were really open, Jinx ranks the true duplicate first as often as the best methods we tested (0.927; a classic research detector 0.914, a link rule 0.936), while raising 1.8% false alarms, against 5.5% for the best existing open-source duplicate-PR bot.

CCS Concepts

• **Software and its engineering → Software evolution**.

Keywords

duplicate pull requests, coding agents, GitHub Actions, code review

1 Introduction

AI coding agents now open hundreds of thousands of PRs on GitHub [4], and duplicated work is a leading reason why they are not merged [1, 2]. Duplicates appear when several agents, or an agent and a person, work on the same issue, or when an agent opens a new PR instead of updating its old one. Each duplicate costs a maintainer a review that did not need to happen.

Existing options leave a gap. GitHub lets maintainers mark duplicates only by hand. Small open-source bots (contrib-guard, Simili, PRSense, PRism, vector-triage, doppelgänger[1]) mostly compare titles and descriptions, and some need a paid LLM API. Research detectors [3, 5–8] are not packaged as maintained tools.

Jinx is built around what a maintainer needs from a warning: it should be right when it speaks, quiet otherwise, explain itself, and run inside the repository's own CI. Our companion study found that, on real open-PR queues, several methods rank duplicates about equally well, but they differ widely in false alarms. Jinx therefore aims for a low false-alarm rate and clear reasons rather than for a new ranking algorithm.

2 How Jinx Works

Fig. 1 shows the five steps.

[1]https://github.com/dannyl1u/doppelganger

1. Read PRs. When a PR is opened, the Action reads the new PR and up to the 200 most recent open PRs through the GitHub API with the workflow's own token: title, description, linked issues, changed files and code changes. The command-line tool reads the same fields from JSON files.

2. Clean text. Agents fill PR descriptions with boilerplate such as checklists and "Generated with …" footers, which makes unrelated PRs look alike. Jinx learns which lines repeat across a repository's PRs and removes them.

3. Compare each pair. Text signals measure how similar the titles and descriptions are (TF-IDF and, by default, a MiniLM embedding that runs locally). Code signals measure shared changed files, overlapping changed line ranges and identical added lines. Context signals record a shared linked issue, a cross-reference, the same author or agent, and the time between the PRs.

4. Score and explain. A logistic regression model, trained on the AIDev benchmark with open-only queues and shipped with the package, turns the signals into a score between 0 and 1. Jinx flags a candidate at 0.5, the point where it flagged about half of the cross-author duplicates in the benchmark with 1.8% false alarms; maintainers can raise or lower it. Every signal that pushes the score up becomes one short sentence.

5. Report. The command-line tool prints a ranked list (Fig. 2). The Action posts one comment on the new PR and updates it on later runs. Jinx never closes, labels or blocks a PR.

3 Using Jinx

A repository adds this workflow:

```
on:
  pull_request_target:
    types: [opened, reopened]
permissions:
  contents: read
  pull-requests: write
  issues: write
jobs:
  jinx:
    runs-on: ubuntu-latest
    steps:
      - uses: <owner>/jinx-pr@v1
        with:
          embed: "true"    # "false" = lexical only
```

The workflow uses `pull_request_target` so that it can comment on PRs from forks. This trigger runs with write permissions, so Jinx never checks out or runs the PR's code: it reads metadata and diffs through the API only. The `embed` input switches the MiniLM model on or off. With it, a PR takes 0.7 s at the median on 4 CPU threads once the model is loaded; without it, 22 ms. Locally, `pip install jinx-pr` provides the `jinx check` and `jinx learn-templates`

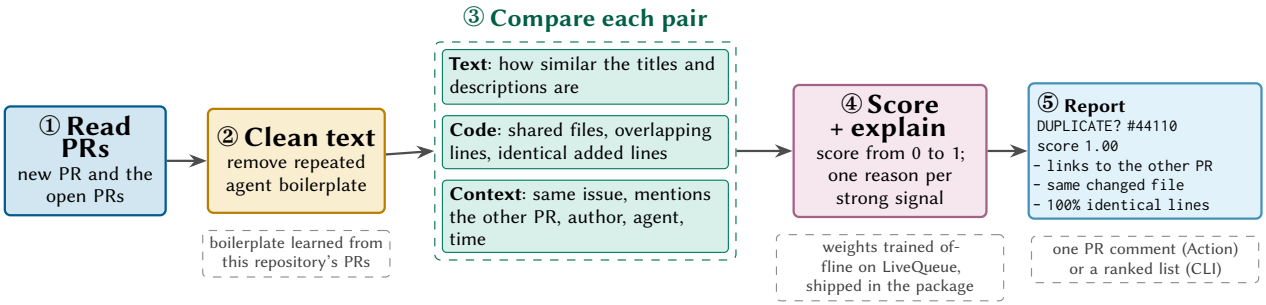


Figure 1: How Jinx works. It compares a new PR with each open PR using text, code and context signals, combines them into one score, and explains each likely match in plain words.

```
$ jinx check new.json --open-queue open.jsonl
PR #44134: checked 19 open PRs in 12 ms.
1 likely duplicate(s) (score >= 0.50).
  DUPLICATE? #44110  score 1.00  refactor(tvp):
     use capture_outputs for output tracing
    - one PR links to the other (#44110)
    - shares 1 of 1 changed files
      (src/transformers/models/tvp/modeling_tvp.py)
    - 100% of changed line ranges overlap
    - 100% of added lines are identical
    - similar title/description after removing
      agent template text (0.92)
  #44133  score 0.05  refactor(poolformer): ...
    - similar title/description (0.50)
    - same author (possible replacement PR)
```

Figure 2: Jinx on a real agent PR from huggingface/transformers, taken from AIDev (lexical mode). The second candidate is listed but not flagged.

Table 1: Cross-author duplicate pairs on real-use queues ($n = 313$); false alarms on 2965 PRs without a duplicate (95% CIs). Underlined: best baseline. Green cell and †: significantly more false alarms than Jinx; ‡: significantly fewer. Last column: Jinx minus the row in Recall@1 points (paired 95% CI).

Strategy	R@1 ↑	R@5 ↑	MRR ↑	False alarms % ↓	Jinx − row, R@1 (pts)
Our tool					
Jinx (ours, default)	0.927	0.990	0.954	1.8 [1.2, 2.5]	–
Jinx without MiniLM	0.917	0.990	0.949	1.9 [1.3, 2.7]	+1.0 [-0.9, +2.7]
Our tool with extra models added					
All signals, LR	0.927	0.990	0.955	1.6 [1.1, 2.2]	-0.0 [-2.0, +1.7]
Jinx + BGE rerank + Qwen-7B	0.936	0.981	0.958	1.4 [0.9, 2.0]	-1.0 [-2.6, +1.0]
Claude Haiku 4.5 + Jinx	0.936	0.990	0.961	0.9 [0.6, 1.3]‡	-1.0 [-2.5, +0.3]
Qwen2.5-14B + Jinx	0.917	0.990	0.949	1.3 [0.8, 2.0]‡	+1.0 [-0.0, +2.0]
Claude Haiku 4.5 alone on Jinx top-5	0.923	0.990	0.956	1.8 [1.1, 2.3]	+0.3 [-2.2, +2.6]
Baselines: link rule, classic detector, existing bot, embedding, text					
Link rule (shared issue or #ref)	0.936	0.978	0.957	1.2 [0.7, 1.7]	-1.0 [-4.3, +2.7]
INTRUDE + time [7, 8]	0.914	0.987	0.945	2.0 [1.3, 3.0]	+1.3 [-0.7, +3.1]
contrib-guard (bot)	0.914	0.981	0.947	5.5 [4.0, 7.0]†	+1.3 [-2.7, +4.7]
TF-IDF [5]	0.911	0.981	0.945	3.5 [2.5, 4.8]†	+1.6 [-2.2, +4.9]
BGE-small embedding	0.907	0.981	0.944	2.3 [1.5, 3.3]	+1.9 [-1.1, +4.6]

commands. For queues longer than 200 PRs the Action looks only at the most recent 200; in our benchmark this cut-off hid the true duplicate in 29 of 1,796 cases (1.6%).

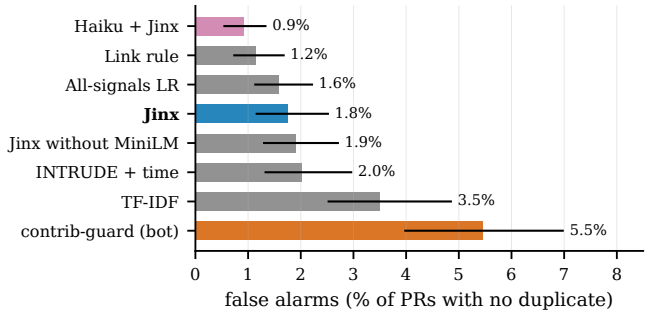


Figure 3: False alarms on PRs with no duplicate, with 95% CIs.

4 Evaluation

We used the AIDev dataset [4]: 1767 pairs of agent PRs that a comment marks as duplicate or superseded, each scored against the PRs that were open in the repository when the newer PR was created, plus 2965 PRs with no duplicate. Weights are trained with five repository-grouped folds for evaluation; the shipped weights are retrained on all data with the same recipe.

Table 1 shows the results for duplicates opened by different authors. Jinx ranks the true duplicate first for 0.927 of the pairs. This is statistically tied with a classic detector (INTRUDE with a time feature [7, 8]), the best existing bot, TF-IDF and a rule that only checks for a shared issue or cross-reference; we do not claim better ranking.

Where Jinx stands out is quietness (Fig. 3). It raises 1.8% false alarms, about a third of the best existing bot's 5.5% (difference +3.7 points, CI +2.2 to +5.1) and half of TF-IDF's 3.5%. Unlike a link rule, Jinx also explains code-level matches (shared files, identical lines), which helps a maintainer check a warning quickly. Teams that want even fewer false alarms can add an LLM check of Jinx's top five candidates: with Claude Haiku 4.5 false alarms fall to 0.9%, at the cost of an API call per candidate.

A replay on AIDev estimates the benefit: had Jinx commented when each cross-author duplicate opened, it would have named the right PR in 46% of cases, including 35% of those that people took more than a day to notice.

5 Limitations

Jinx often misses duplicates that change different files to reach the same goal (0.720 Recall@1 on such cross-author pairs). Labels come from PR comments; an LLM audit judged 84.8% of the cross-author labels to be true duplicates or replacements, and a human check is pending. We have evaluated Jinx on historical data only; a run on live repositories is future work.

6 Availability

Jinx is open source under the MIT licence. The tool, the benchmark and all scripts will be public with the camera-ready version, together with a short demo video.

References

[1] Khairul Alam, Saikat Mondal, and Banani Roy. 2026. Why Are AI Agent Involved Pull Requests (Fix-Related) Remain Unmerged? arXiv:2602.00164. arXiv:2602.00164

[2] R. Ehsani et al. 2026. Where Do AI Coding Agents Fail? An Empirical Study of Failed Agentic Pull Requests in GitHub. arXiv:2601.15195 MSR 2026.

[3] Jing Jiang, Qiudi Li, Jin Guo, Zhenrui Li, Li Zhang, and Ge Li. 2023. DupHunter: Detecting Duplicate Pull Requests in Fork-Based Development. *IEEE Trans. Softw. Eng.* (2023). doi:10.1109/TSE.2023.3235942

[4] Hao Li et al. 2026. AIDev: Studying AI Coding Agents on GitHub. arXiv:2602.09185

[5] Zhixing Li, Gang Yin, Yue Yu, Tao Wang, and Huaimin Wang. 2017. Detecting Duplicate Pull-Requests in GitHub. In *Proc. Internetware.* doi:10.1145/3131704.3131725

[6] Zhifang Liao, Yuanyuan Liu, Yuqi Lan, and Yan Sun. 2024. DupLLM: Duplicate Pull Requests Detection Based on Large Language Model. In *Proc. APSEC.* doi:10.1109/APSEC65559.2024.00023

[7] Luyao Ren, Shurui Zhou, Christian Kästner, and Andrzej Wąsowski. 2019. Identifying Redundancies in Fork-based Development. In *Proc. SANER.* doi:10.1109/SANER.2019.8668023

[8] Qingye Wang, Bowen Xu, Xin Xia, Ting Wang, and Shanping Li. 2019. Duplicate Pull Request Detection: When Time Matters. In *Proc. Internetware.* doi:10.1145/3361242.3361254